

Container Orchestration

INSISN – Integração de Sistemas e Serviços na Nuvem

António Barros

Departamento de Engenharia Informática

Introduction to Container Orchestration

Why Orchestrate? Moving Beyond Single-Host Setups

The Single-Host Limitation:

- Tools like docker run or single-host Docker Compose work well in development.
- Single Point of Failure (SPOF): If the physical host fails, all services go down.

Production Requirements:

- **High Availability (HA):** Ensuring zero-downtime operations.
- **Horizontal Scaling:** Seamlessly scaling container instances across multiple machines.
- **Self-Healing:** Automatically recovering from hardware or application failures.
- **Resource Allocation:** Efficiently balancing workloads across compute resources.

Fundamental Cluster Concepts

What is a Cluster?

A collection of interconnected physical or virtual machines working together as a single system.

Infrastructure Abstraction:

- Aggregates individual compute resources (CPU, Memory, Storage, Network).
- Developers deploy applications to the *cluster*, not to specific individual machines.
- The orchestrator handles placement, isolation, and workload distribution.

Cluster Anatomy

A cluster is composed of **master** and **worker** nodes.

Master Nodes (Control Plane):

- Act as the *brain* of the cluster.
- Responsible for scheduling, maintaining cluster state, and handling API requests.
- Do not typically run user workloads in production environments.

Worker Nodes:

- Act as the *muscle* of the cluster.
- Receive instructions from the control plane to execute containers/tasks.
- Report local status and health metrics back to the manager nodes.

The Concept of Desired State

Declarative vs Imperative Management:

- *Imperative*: "Run container X on server Y right now."
- *Declarative*: "Ensure 5 replicas of application X are running at all times."

Desired State vs Actual State:

- **Desired State**: The blueprint provided in your deployment configuration.
- **Actual State**: The live, real-time condition of the cluster.

Goal of the orchestrator: Continuously align actual state with desired state.

The State Reconciliation Loop

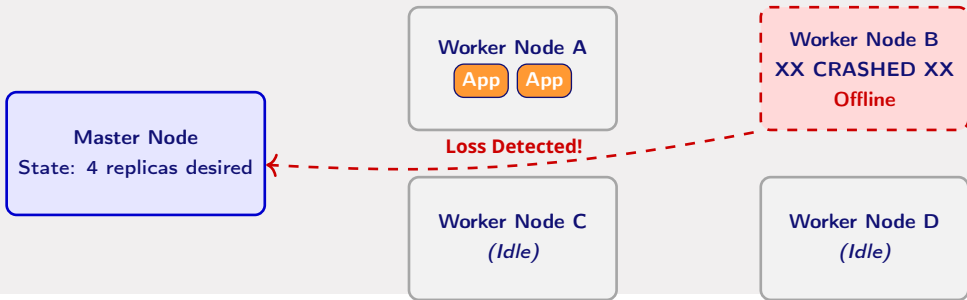
State reconciliation is achieved through a continuous 3-step loop that aligns the cluster's **actual state** with its **desired state**:

1. **Monitor:** Continuously poll the actual state of nodes and tasks.
2. **Observe:** Detect discrepancies between desired state and actual state.
3. **Act:** Trigger corrective actions to eliminate differences.

Reconciliation Scenario: Node Failure

Real-World Example (Part 1: State Discrepancy)

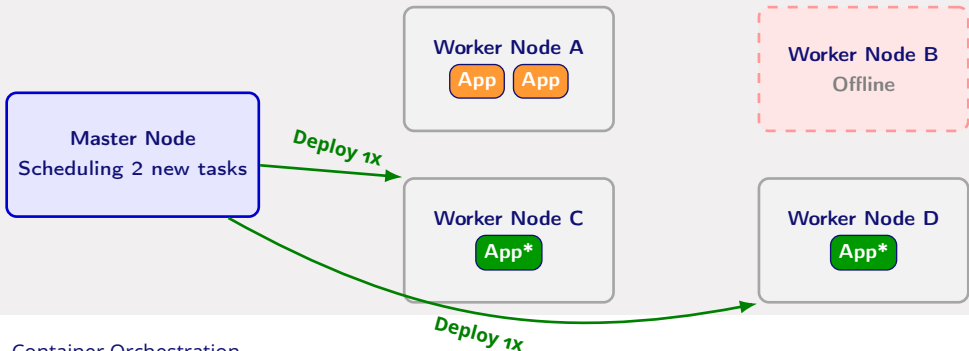
- **Desired State:** 4 container instances running.
- **Actual State:** Node B crashes → Active instances drop from 4 to 2.



Reconciliation Scenario: Self-Healing Action

Real-World Example (Part 2: State Recovery)

- **Corrective Action:** Master schedules 2 new instances on healthy nodes.
- **Outcome:** Cluster restores desired state (4/4 instances active).



Docker Swarm Architecture and Internal Mechanics

What is Docker Swarm?

Native Orchestration Engine:

- Built directly into the Docker Engine (*Swarm Mode*).
- Zero external dependencies or complex third-party installations.

Key Advantages:

- **Simplicity:** Activated with a single command (`docker swarm init`).
- **Familiar CLI:** Uses standard Docker CLI tools and Compose file formats.
- **Built-in Security:** Mutual TLS (mTLS) enabled by default for node authentication and encryption.

Enabling Swarm Mode

The `docker swarm init` command is the entry point for turning a standalone Docker host into a cluster control plane.

It bootstraps the Raft consensus store, establishes a root CA for mTLS security, and exposes the cluster's management IP so other nodes can join securely.

Initialization Command (Master)

```
$ docker swarm init --advertise-addr 192.168.1.10
```

Command Breakdown

- **docker swarm init:** Converts standalone engine to Manager node and initializes the Raft state database.
- **--advertise-addr:** Defines the manager's IP (192.168.1.10) exposed to the cluster for control plane traffic.

Managing Swarm Join Tokens & Node Roles

Token-Based Role Identification

- In Docker Swarm, security and topology are managed via tokens.
- The token provided during join time acts as a cryptographic identity that determines whether a node enters the cluster as a **Worker** or a **Manager**.

Retrieving Join Tokens (Run on a Manager Node)

```
# Retrieve token for joining as a Worker
$ docker swarm join-token worker

# Retrieve token for joining as a Manager
$ docker swarm join-token manager
```

Managing Swarm Join Tokens & Node Roles

Key Concepts and Best Practices

- **Role Embedded in Token:** The token string contains a hash that encodes the node role:
 - ▶ Worker: SWMTKN-1-...-worker
 - ▶ Manager: SWMTKN-1-...-manager
- **Least Privilege Principle:** Always join nodes as Workers by default. Promote to Manager only when control plane redundancy is needed.
- **Token Rotation for Security:** Tokens should be rotated periodically or after security incidents to prevent unauthorized nodes from joining:
 - ▶ `$ docker swarm join-token --rotate worker`

Joining Nodes to the Cluster

Expanding the Swarm Architecture

- Once the Manager initializes the cluster and issues mTLS certificates, new machines can safely join the topology.
- By passing the generated secret token to the `docker swarm join` command, nodes authenticate with the control plane and establish an encrypted connection over port 2377.

Joining Nodes to the Cluster (Example)

Join Command (Manager)

```
$ docker swarm join --token SWMTKN-1-49ty... 192.168.1.10:2377
```

Command Breakdown

- **docker swarm join:** Instructs the local Docker engine to join an existing Swarm cluster as a worker or manager.
- **--token <secret-token>:** Provides a cryptographically signed token that authenticates the node and determines its role (*Worker vs Manager*).
- **192.168.1.10:2377:** The IP address of an active Manager node and the default target port for cluster control plane traffic.
- **mTLS Handshake:** The manager verifies the token, issues a node-specific TLS certificate, and establishes a secure overlay channel.

Swarm Internal Architecture: Control Plane Components

Cluster Topology Overview

- **Manager Nodes:** Control plane operations, state storage, and orchestration.
- **Worker Nodes:** Execute tasks (containers) assigned by managers.

Internal Components of a Manager Node

- **API Server:** Entry point for HTTP/REST requests (Docker CLI / Swarm API).
- **Raft Store:** Distributed, strongly consistent state database.
- **Scheduler:** Assigns pending tasks to suitable worker nodes.
- **Allocator:** Allocates IP addresses and network resources to tasks.
- **Dispatcher:** Coordinates workers and monitors task execution status.

Node Communication and Consensus: Raft Protocol

- **Maintaining State Consistency:**

- ▶ Manager nodes use the **Raft Consensus Algorithm** to synchronize cluster state.
- ▶ Guarantees that all managers share an identical view of configurations, secrets, and task statuses.
- ▶ Prevents "split-brain" scenarios through leader election.

- **Understanding the Quorum:**

- ▶ A **Quorum** is the minimum majority of active managers required to make decisions.
- ▶ **Quorum Formula:**

$$\text{Quorum} = \left\lfloor \frac{N}{2} \right\rfloor + 1$$

- ▶ If active managers drop below the Quorum, the cluster stops accepting configuration changes to prevent data corruption.

Fault Tolerance and Manager Topology

- **Fault Tolerance Capacity:**

- ▶ For a cluster with N manager nodes, the maximum allowed node failures F is:

$$\text{Fault Tolerance } (F) = \left\lfloor \frac{N - 1}{2} \right\rfloor$$

- **Why Always Use an Odd Number of Managers?**

- ▶ **3 Managers:** Quorum = 2 → Tolerates **1** failure.
- ▶ **4 Managers:** Quorum = 3 → Still tolerates only **1** failure!
- ▶ **5 Managers:** Quorum = 3 → Tolerates **2** failures.

Adding an even number (e.g., 4th node) increases network overhead and failure risk *without* improving fault tolerance.

Swarm Abstraction Hierarchy: Motivation

Why Abstraction Matters in Distributed Systems

In a single-host environment, we manage individual containers directly. However, in a multi-node cluster, managing containers manually leads to operational bottlenecks and consistency issues.

- **Decoupling Intent from Infrastructure:** Operators define *what* application state is needed, not *where* or *how* individual containers execute.
- **The Need for Atomic Work Units:** Orchestrators require immutable, predictable units of execution to scale, reschedule, and update workloads safely without manual intervention.
- **Two-Tier Abstraction Model:** Docker Swarm introduces a clean hierarchy: **Services** (declarative management) and **Tasks** (runtime execution).

Services vs Tasks: Core Abstractions

- **Service (Declarative Definition):**

- ▶ High-level blueprint of the desired application state.
- ▶ Defines parameters such as image tag, open ports, network attachments, environment variables, and replica count.
- ▶ Remains persistent throughout the application lifecycle.

- **Task (Atomic Execution Unit):**

- ▶ Represents an individual container allocation assigned to a specific worker node.
- ▶ **Immutable Nature:** Tasks are never restarted or repaired in-place. If a task fails or its node crashes, the Manager destroys it and instantiates a brand new task.

- **The 1 : N Relationship:**

- ▶ 1 Service acts as a parent controller that spawns and manages N replica Tasks distributed across the cluster.

Deploying Services and Managing Tasks

Example: Creating a Service with 3 Replicas

Deploying a workload relies on the `docker service create` command:

```
$ docker service create --name web-api \  
  --replicas 3 \  
  --publish published=8080,target=80 \  
  nginx:alpine
```

Execution and Allocation Result:

- **Service Created:** `web-api` (a logical entity registered in the Raft Store).
- **Tasks Spawned:** `web-api.1`, `web-api.2`, and `web-api.3` are dispatched across available, healthy worker nodes.

Deploying Services and Managing Tasks

Inspecting Active Tasks Across Nodes

To verify how tasks are distributed and monitor their real-time execution status across cluster nodes, use the **docker service ps** command:

```
$ docker service ps web-api
```

ID	NAME	IMAGE	NODE	DESIRED STATE
x1y2z3...	web-api.1	nginx:alpine	node-01	Running
a4b5c6...	web-api.2	nginx:alpine	node-02	Running
d7e8f9...	web-api.3	nginx:alpine	node-03	Running

Task Distribution and Scheduling Mechanics

When a service is created or scaled, the Swarm Scheduler determines the optimal node for each task through a sequential filtering process:

1. **Resource Constraints:**

- ▶ Checks CPU and Memory reservations/limits against available node resources.
- ▶ Filters out nodes with insufficient capacity.

2. **Placement Constraints:**

- ▶ Applies declarative rules (e.g., node roles, custom labels, engine attributes) to restrict task placement.

3. **Spread Strategy:**

- ▶ Balances tasks evenly across eligible worker nodes to ensure high availability and prevent resource exhaustion.

Task Distribution and Scheduling Mechanics (Example)

Operators can steer task placement by tagging nodes with custom labels and applying placement constraints during service creation.

Targeting Specific Nodes via Node Labels

```
# 1. Tag a specific worker node with a zone label
$ docker node update --label-add zone=prod-north worker-01

# 2. Deploy a service restricted by label and resource reservations
$ docker service create --name db-service \
  --constraint 'node.labels.zone == prod-north' \
  --reserve-memory 512M \
  redis:alpine
```

State Reconciliation: Fundamental Rules

The Continuous Self-Healing Rule

Docker Swarm operates on a declarative model where the orchestrator continuously compares the *Desired State* (Raft Store) against the *Actual State* (Reported by Workers).

- **Heartbeat & Failure Detection:**

- ▶ Worker nodes send periodic heartbeats to Managers over gRPC (port 2377).
- ▶ If a node fails to report within a timeout period, its status transitions to Down.

- **Task Immutability Principle:**

- ▶ Tasks assigned to a failed node are marked as DEAD or UNKNOWN.
- ▶ Tasks are **never migrated**; the Scheduler creates brand-new Task objects on surviving healthy nodes to restore the desired replica count.

- **Reconciliation Trigger:**

- ▶ Action occurs automatically whenever $\text{Actual Replicas} < \text{Desired Replicas}$.

State Reconciliation in Practice: Node Failure

Scenario: Physical Worker Failure

Consider a service running 6 replicas across 3 worker nodes. Worker 2 loses power unexpectedly:

1. **State Discrepancy:** Node wrk-02 goes Down with 2 active replicas. Actual running count drops to 4/6.
2. **Automatic Recovery:** Manager detects missing heartbeats, terminates lost tasks, and schedules 2 new replacement tasks on Node wrk-01 and Node wrk-03.

```
$ docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS
node1 *	mgr-01	Ready	Active	Leader
node2	wrk-01	Ready	Active	
node3	wrk-02	Down	Active	<-- Triggers reschedule
node4	wrk-03	Ready	Active	

Conceptual Workflow: Command to Execution

End-to-End Lifecycle of a Service Request:

1. **User Request:** Admin executes `docker service create`.
2. **API and Store:** Manager API Server receives request and writes state to Raft Store.
3. **Orchestrator and Scheduler:**
 - ▶ Orchestrator breaks Service into individual Task objects.
 - ▶ Scheduler evaluates node capacity and assigns Tasks to Worker nodes.
4. **Dispatcher and Execution:**
 - ▶ Dispatcher sends assigned Tasks to Worker Node agents.
 - ▶ Worker Node Agent invokes Docker Engine to pull image and start container.